Machine-Learning-Assisted Adaptive Chunk Scheduling for Multiprocessing Workloads

Operating Systems Review 2 Methodology and Partial Implementation

Sashwath J 24BAI0059

This prototype sends independent CPU tasks to a Linux process pool. Fixed chunks of four tasks are the reference. Two alternatives are implemented: a decision tree changes chunk size using live measurements, while a cost-balanced dispatcher groups tasks using known metadata. The results show different strengths and costs. The combined method proposed for the next review is not yet implemented.

1 Problem and proposed method

A fixed chunk size gives each worker the same number of tasks per batch even when task cost varies. Small chunks can create dispatch overhead; large chunks can leave one worker with a long final batch. The project tests two responses to this problem. The decision-tree path observes early execution and predicts one of four chunk sizes. The cost-balanced path uses available task metadata to make batches of roughly similar estimated cost, then sends longer batches first. The Linux kernel still schedules the worker processes; this project changes user-space task dispatch, not kernel code.

The next question is whether a combined scheduler can use the tree's workload signal and cost balancing without the current tree path's repeated round barrier. This is a testable design, not a measured result. Machine-learning-assisted task scheduling and adaptive self-scheduling already exist in the literature (Qawasmeh et al., 2016; Booth & Lane, 2022; Nookala et al., 2024).

1.1 What the scheduler controls

The unit of work is one independent CPU task. A task contains an amount of work, a deterministic seed or starting integer, and a kernel type. A chunk is a group of these tasks sent to one process-pool worker as one batch. Changing the chunk changes how often the parent submits batches and how much work can be left in a worker's final batch. It does not change the Linux CPU time-sharing policy. The operating system still decides when each worker process runs; the project decides which application tasks are grouped and submitted to those processes.

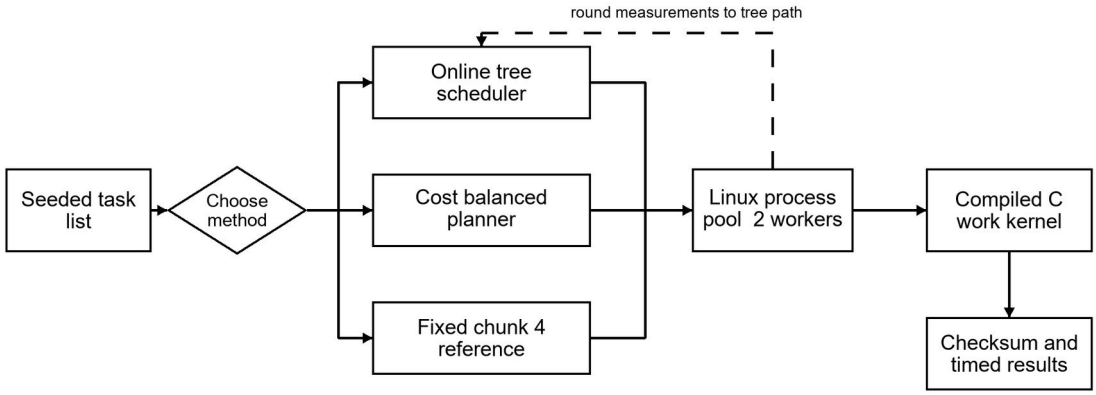
With 192 tasks, fixed chunk 4 forms 48 four-task batches. A small chunk gives the pool more chances to distribute uneven work but creates more submissions and result handling. A large chunk reduces that administrative work, yet a costly task inside a late batch can determine when the entire job finishes. The correct choice therefore depends on both task variability and scheduling overhead. The benchmark tests that tradeoff at the complete application boundary instead of assuming that fewer batches must always be faster.

1.2 Scope of the claim

The work has two implemented alternatives to the fixed-4 reference: a trained tree that selects a later chunk from runtime measurements, and a separate metadata-based cost balancer. Both execute the same generated tasks and must return the same checksum. The planned combined scheduler is a research direction, not a completed result. This distinction matters because the measured cost-balanced improvement cannot be presented as an ML improvement, and a model's ability to predict a profiling label cannot be treated as proof of lower wall time.

2 System architecture

Figure 1 Implemented user space scheduling architecture. The dashed line shows runtime feedback used only by the tree path.



The controller runs in Python 3.12.3 on Ubuntu under WSL2. It creates tasks, chooses fixed, tree or costbalanced dispatch, and sends batches to two worker processes. Each worker calls the compiled C11 CPU kernel through ctypes. The parent records timing and checksums after results return. Model fitting is offline; the measured run loads an already trained tree.

2.1 Where time is spent

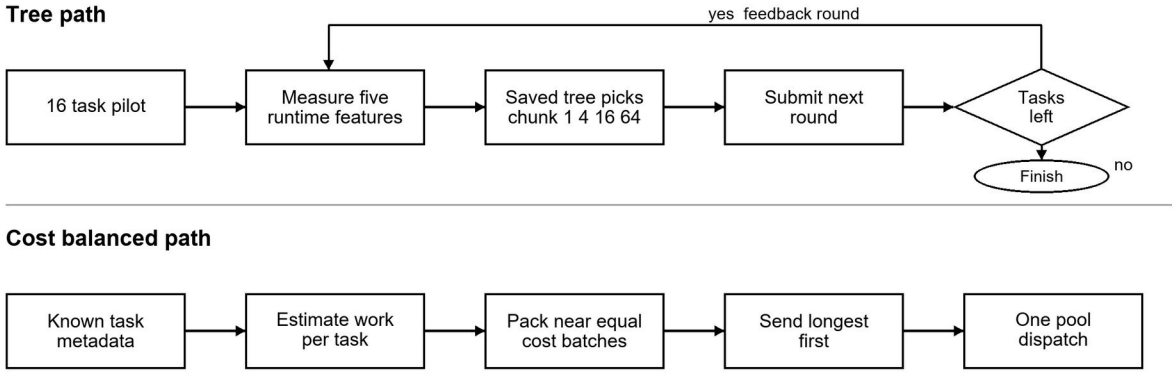
The parent process constructs the task list, selects the scheduling method, creates a pool of two worker processes, submits batches, receives results, and closes the pool. Workers load the compiled C library through ctypes when needed. In the synthetic kernel, each task performs a deterministic integer loop; in the prime-scan kernel, it counts primes in its assigned integer range. The returned value lets the parent compare scheduler outputs without depending only on timing. The C code makes the task computation substantially different from a Python loop, but Python still owns scheduling and data movement.

The main wall-time timer begins before method-specific planning or pool creation and ends after results return and the pool closes. That includes the costs a caller experiences when invoking this program. Offline model fitting is performed separately and is not hidden inside each timed invocation. Pool startup, dispatch, round waiting, native computation, result collection, and teardown can all influence the full time. Consequently, a microsecond-scale tree decision can coexist with a slower overall tree run.

3 Major modules and workflows

Module	Implemented responsibility	Main output
Task generator	Recreates 192 CPU tasks from a workload name and seed.	Task list and seed
Native worker	Executes synthetic iteration work or prime counting in C.	Checksum and service time
Tree trainer	Profiles candidate chunks, labels fastest choices, fits a depth-3 tree.	model.json
Module	**Implemented responsibility**	**Main output**
Tree scheduler	Runs a pilot, measures feedback, predicts the next chunk per round.	Decision trace and timing
Cost balancer	Estimates task cost, packs batches, sorts them longest first.	Batches and timing
Evaluator	Times matched runs and audits checksums and summary medians.	raw.csv and summary.csv

Figure 2 The two implemented scheduling paths. The tree waits at feedback rounds; cost balancing prepares batches once.



The generator first creates the same seeded task list for each method. Offline training profiles candidate chunks and saves model.json; the evaluator later times each method on matched inputs, checks output equality, and recomputes the paired statistics. These training and evaluation steps run outside the two dispatch paths shown in Figure 2.

Figure 2a Workflows for the remaining implemented modules. Training is offline; evaluation uses matched tasks and checksums.

Task generator	Workload + seed	→	Build 192 task specifications	→	Reuse identical list for each method
Native worker	Pool sends a batch	→	ctypes calls compiled C kernel	→	Return checksum and service time
Offline tree trainer	Profile chunks 1, 4, 16, 64	→	Label fastest and fit depth-3 tree	→	Save model.json
Evaluator	Same seed; shuffled method order	→	Record time and verify checksums	→	Audit and summarize paired ratios

3.1 The task and worker path

The generator in os_sched/workload.py creates frozen task records from a workload name and seed. For the five general patterns, the paired review uses 192 tasks per record and passes either 1.5 million base C iterations or a 3,000-integer prime range. The late-heavy generator in os_sched/straggler_workload.py places one 40-times-cost task among the last eight task positions; the remaining tasks fluctuate between 0.9 and 1.1 times base work. This is intentionally constructed to test a late straggler, not presented as an observed production workload.

The parent converts the task list into batches. Each worker applies the chosen C function to every task in its batch, sums the deterministic results, and records the batch's start, end, and service time. The parent then sums batch checksums. If fixed, tree, and balanced paths disagree on that final checksum for the same seed, the comparison is rejected. The test suite also checks generator determinism and the scheduling modules. A matching checksum is an output-equivalence check for these runs, not a proof that every possible task ordering is correct.

3.2 How the tree learns a chunk label

Offline profiling first runs a pilot at a live-equivalent decision boundary. For each of five workload types, six training seeds and three current chunks (1, 4, and 16) produce 90 fitted records. Four separate holdout seeds produce 60 more records. After the pilot, the same next 64 tasks are tested with candidate chunks 1, 4, 16, and 64. Each candidate is measured twice, and candidate order is shuffled in each pass to reduce systematic order effects. The label is the chunk with the lowest candidate median; if several candidates are within 3 percent of the fastest, the larger chunk is preferred to avoid needless small-batch work.

The five inputs saved with each label are mean measured task service time, normalized spread of batch service times, completed tasks per second in the pilot, the current chunk, and the fraction of tasks still unstarted. The model is a depth-3 CART-style classifier with at least five records per leaf. Its JSON file stores threshold tests and leaf labels, so a live run can load the fitted tree without an external ML package. On the separate 60-row holdout, 44 labels match exactly, or 73.3 percent. That number describes agreement with profiling labels only; it says nothing by itself about the full application's speed.

3.3 What happens during a live tree run

The tree path begins with chunk 4. In each round it selects up to current chunk multiplied by workers multiplied by two unstarted tasks; the first two-worker round therefore submits 16 tasks. The pool call completes that round before the parent computes throughput, batch spread, mean task service time, and the remaining fraction. The saved tree predicts the chunk for the next round. A decision event records the old and new chunks, reason, elapsed round time, and inference time. The live evidence shows a 4-to-16 prediction, but that trace is one demonstration rather than an aggregate performance estimate.

Waiting for a round to finish provides coherent feedback, but it is also a synchronization barrier. A fast worker may be idle while a slower batch determines when the parent can decide again. That limitation is visible in the distinction between the tree's low batch p95 and its higher full wall time on the late-heavy case. The program also contains a guarded-tree variant that can reduce a predicted chunk after a measured task-cost rise; that earlier variant is not one of the three methods in the current paired Review 2 table.

3.4 What cost balancing does instead

The balanced path estimates synthetic cost from iterations and prime cost from range length times an approximate square-root factor for integer magnitude. It divides total estimated work by a target of 32, groups adjacent tasks until the target-cost or 32-item limit is met, then sorts batches from highest to lowest estimated cost. Thirty-two is a target, not a promise of exactly 32 batches.

The pool receives the prepared list once, starting longer estimated batches early. The live late-heavy run places task 187 in dispatch position 0. This is not a learned model and cannot react to work whose cost is unknown or badly estimated. It complements the online tree, but a hybrid is still unmeasured.

4 Dataset and collection procedure

The inputs are reproducible generated CPU workloads, not an external dataset or collected production trace. Every timing group uses the same task list for all three methods. Five general patterns each have five seeds (7001-7005); the late-heavy pattern has ten seeds (11001-11010). Each group contains 192 tasks on two workers. Training uses different seeds (11-16), the holdout uses 101-104, and the paired benchmark uses the later seeds. This separation limits direct train/test leakage.

Pattern	How the 192 tasks are generated	Why included
Uniform	Each synthetic task has 1.5 million C iterations.	Low variability control
Moderate	Each task scales 1.5 million iterations by a seeded factor from 0.5 to 1.5.	Ordinary uneven work
High	70 percent are about 0.2 times and the remainder about 2.87 times base work.	Strong size variation
Phase shift	First half is light; second half mixes very light and heavy work.	Distribution change during a run
Prime scan	C counts primes in successive 3,000-integer ranges.	Non-synthetic computational kernel
Late heavy	One of the last eight tasks has 40 times base iterations; others vary 0.9-1.1 times.	Rare delayed straggler

The training table has 90 fitted rows and 60 held-out rows. Each row is a measurement at a liveequivalent decision boundary. Its five predictors are mean task service time (average worker C time per task), batch spread ((maximum minus minimum batch service time) divided by maximum), throughput (tasks

completed divided by round seconds), current chunk size, and fraction of tasks remaining. The label is the fastest median candidate among chunks 1, 4, 16 and 64; choices within 3 percent of fastest prefer the larger chunk. These sizes span one task through coarse batching, rather than being arbitrary output classes. The held-out exact label accuracy is 73.3 percent (44 of 60); this is a classification result, not a runtime speedup.

Collection is deterministic from the generator and saved seeds. The earlier full run saved 4,800 task records in results_final_linux/dataset.csv. The paired study stores each group seed, method order, full timings and checksum in raw.csv; its generator code recreates the exact task list. No personally identifiable or third-party data is used.

4.1 What is saved and what can be reconstructed

The project does not import a public scheduling dataset. Inputs are generated deterministically from workload definitions and seeds. The earlier full training run saves task-level rows in results_final_linux/dataset.csv and the fitted tree in model.json. The later paired review uses different seeds and saves one timing row per method and seed in results_paired_both_review/raw.csv. The compact summary.csv is derived from those raw rows, while manifest.json records the workload count, methods, timing boundary, and host scope. The raw rows are the primary evidence for the reported medians; the summary is a convenience for reading them.

Training, holdout, and paired evaluation use disjoint seed ranges: 11-16 for fitting, 101-104 for label holdout, 7001-7005 for the five general benchmark patterns, and 11001-11010 for late-heavy trials. A seed fixes task generation and the method-order shuffle; it does not freeze operating-system timing. Fresh elapsed seconds can move with host load, so the report keeps the saved paired experiment as its numeric reference. Every result is tied to a generated pattern rather than a claim about an unseen production trace.

4.2 Why these workload patterns matter

Uniform tasks are a control for dispatch overhead: there is little imbalance to correct. Moderate and high patterns introduce different degrees of cost variation without changing the task count. Phase shift puts different work distributions in the two halves of a run, which tests whether online feedback is useful after conditions change. Prime scan substitutes a genuine prime-counting C kernel for the synthetic iteration loop, while the deliberately late-heavy pattern exposes the cost of leaving an unusually expensive task near the end. Together they vary one important scheduling property at a time, but they still cover only a narrow part of real multiprocessing work.

5 Tools technologies and algorithms

Execution used Ubuntu on WSL2, Python 3.12.3 for the controller and multiprocessing pool, GCC 13.3 for the C11 work kernel, ctypes for Python-to-C calls, and the Python standard library for a custom CART-style decision tree and CSV/JSON output. The fixed reference groups four tasks. The tree is depth 3 with minimum leaf size 5. The cost method sets target cost to total estimated cost divided by 32, greedily packs adjacent tasks up to 32 items, sorts batches by estimated cost and dispatches them once. Synthetic cost is estimated from the known iteration count. Prime cost uses work multiplied by an approximate square-root scale of the number range. The cost estimate is task metadata, not a learned prediction.

5.1 Software and file roles

The native/work_kernel.c file defines the two deterministic CPU functions. The task generators live in os_sched/workload.py and os_sched/straggler_workload.py. os_sched/scheduler_v2.py contains the fixed and online-tree paths, timing capture, and batch-result fields. os_sched/tree.py contains the small decision-tree implementation; os_sched/scheduler_v4.py contains cost estimation and balanced batching. The paired experiment and its independent audit are in os_sched/paired_both_review.py and os_sched/verify_paired.py. These files separate task computation, scheduling policy, and evaluation rather than treating a chart or screenshot as the implementation.

Bash and GCC compile the C library, while Python 3.12.3 creates multiprocessing workers and handles CSV and JSON records. The code uses the Python standard library for the tree rather than scikit-learn, so model fitting and live inference have no additional Python package dependency. Linux under WSL2 is the reported host. The process-pool behavior and its overhead are part of the experiment; this is not a Linux kernel patch or a direct comparison of kernel schedulers.

6 Experimental plan and measures

The planned comparison is fixed chunk 4 versus online decision tree versus cost-balanced 32 under matched tasks. For each workload and seed, method order is seeded and shuffled. The primary measure is the median across seeds of each method's wall-time ratio to fixed 4 on the same seed. A ratio below 1 means lower full elapsed time; 0.870 means about 13 percent less time. Wall time starts before batch planning or pool creation and stops after results return and pool teardown. Offline model training is excluded from a test run but reported separately. This boundary measures application-level execution, not a kernel scheduling event.

Measure	Definition	Interpretation
Full wall time	Time from scheduling start through planning, pool startup, dispatch and teardown.	Primary user-visible latency; lower is better.
Paired ratio	Method time divided by fixed-4 time for the same input seed; median of ratios.	Below 1 faster; above 1 slower than baseline.
Batch p95	Per-run 95th percentile from batch submission to worker completion.	Shows slow-tail behavior; not the same as total wall time.
Startup	Time from run start until worker pool is ready.	Part of full wall time.
Checksum	Sum of deterministic C results for all 192 tasks.	Equality checks output correctness across methods.

The benchmark has 35 matched input groups and 105 timed method runs. Its saved audit recomputes all summary medians, checks method coverage and order, and confirms equal checksums in each group. The sample is exploratory on one WSL2 machine with two logical CPUs. No confidence interval or crossmachine generalization is claimed.

6.1 Pairing and metric calculation

For each of the 35 workload-and-seed groups, the code constructs one task list and runs fixed 4, the loaded tree, and balanced 32 on that same list. Method order is deterministically shuffled within each group to avoid always giving one method the same position. The experiment rejects a group if the three final checksums differ. It produces 105 raw method rows and 18 workload-by-method summary rows. The audit independently checks row count, method coverage and order, checksum equality, and recomputed medians. No timed run is replaced by a made-up value when explaining the chart.

The primary statistic for method M on one workload is median over seeds of time(M, seed) divided by time(fixed 4, same seed). Thus 0.870 means that the typical paired ratio is about 13 percent below the fixed-4 reference; 1.245 means about 24.5 percent above it. A ratio of displayed median times is not generally equal to the median of paired ratios. The latter retains seed matching and is the value used for the comparison in Table 1 and Figure 3.

6.2 Why latency measures need care

Batch p95 is calculated from the response times of batches within a run, from submission to completion, and the report displays the median of those per-run p95 values across seeds. It describes a slow portion of batch responses, not the elapsed time of the whole job. Startup time describes the period before workers are ready; queue wait and worker service can also be inspected in raw.csv. These components are useful for diagnosis, but their medians or percentiles cannot simply be added to reconstruct the full wall-time median.

The full wall boundary starts before planning and pool startup and ends after all results return and the pool closes. This prevents a method from appearing fast merely because its setup or teardown was timed elsewhere. Offline tree training is excluded from an invocation because the saved model is reused, but its cost and data collection are documented separately. All measurements remain exploratory: five seeds for each general pattern, ten for late-heavy, one WSL2 host, and two workers. The report does not attach confidence intervals or assert that these values will transfer to another machine.

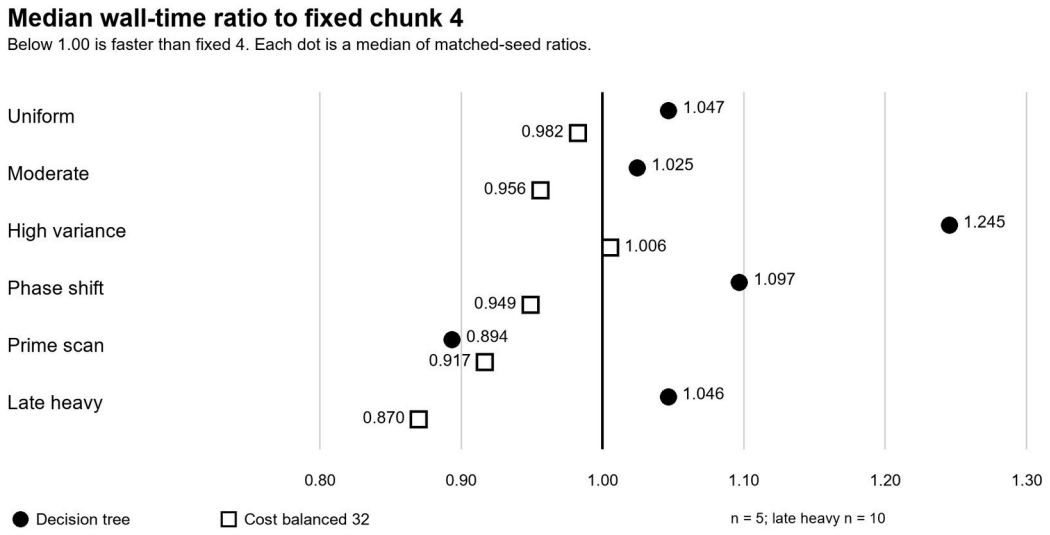
7 Measured comparison and interpretation

Table 1 Full wall time and median paired ratio by workload

Workload	n	Fixed 4 s	Tree s / ratio	Balanced s / ratio
uniform	5	0.365	0.385 / 1.047	0.358 / 0.982
moderate	5	0.403	0.399 / 1.025	0.367 / 0.956
high	5	0.366	0.456 / 1.245	0.364 / 1.006
phase shift	5	0.372	0.411 / 1.097	0.368 / 0.949
prime scan	5	0.235	0.212 / 0.894	0.201 / 0.917
late heavy	10	0.496	0.522 / 1.046	0.432 / 0.870

Values are medians over five seeds per general workload and ten late-heavy seeds. The ratio is the median of within-seed ratios, so it need not equal a ratio of the two displayed median seconds. This matters for moderate and prime scan, where comparing the seconds alone could give a different ranking from the paired statistic.

Figure 3 Paired wall-time ratios calculated directly from the saved benchmark summary. The vertical line is the fixed-4 baseline.



The tree improves the paired ratio on prime scan (0.894) but is slower than fixed 4 on uniform, moderate, high, phase-shift and late-heavy tasks. Its high-variance ratio is 1.245, about 24.5 percent more time. A plausible contributor is the repeated round barrier in the current implementation: each model decision

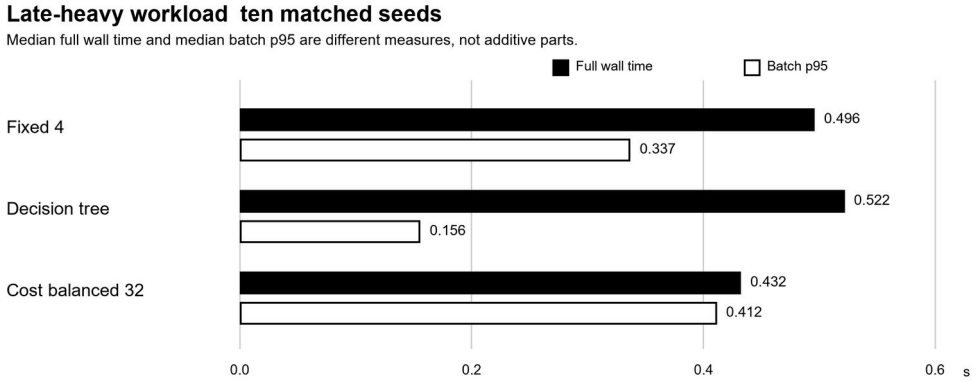
waits for a round's submitted batches to finish. Prediction itself takes only microseconds in the live trace, but the full dispatch pattern can still be slower. The present data do not isolate a single cause.

Cost balancing is strongest on the intended late-heavy pattern: median paired ratio 0.870, or about 13 percent less full time than fixed 4; median wall times are 0.432 s versus 0.496 s. It also improves the paired ratio on uniform, moderate, phase-shift and prime scan, but is slightly slower on high variance (1.006). This is evidence for a targeted benefit, not universal superiority. It does not make the cost method machine learning.

Table 2 Late heavy diagnostics Ten matched seeds

Method	Wall s	Paired ratio	Batch p95 s	Startup s
Fixed 4	0.496	1.000	0.337	0.005
Decision tree	0.522	1.046	0.156	0.005
Cost balanced 32	0.432	0.870	0.412	0.006

Figure 4 Late-heavy full wall time and batch p95 from ten matched seeds. The two measures describe different timing boundaries.



A lower batch p95 does not guarantee lower total wall time. In the late-heavy group the tree has the lowest batch p95 (0.156 s) but slower total time (0.522 s). The cost path starts its estimated heavy batch early; in the live demonstration, task 187 moved to dispatch position 0. Its batch p95 is higher (0.412 s) while total time is lower. The single live run itself is not used as the ten-seed effect estimate; its exact elapsed times can change with system load.

7.1 Reading the workload-level results

The tree's best paired ratio in the present study is on prime scan, 0.894, while its high-variance ratio is 1.245. That uneven outcome is important: the model may choose useful chunks under one cost pattern yet lose time through repeated round boundaries under another. Cost balancing's strongest intended result is late-heavy at 0.870. Its other paired ratios are below one on uniform, moderate, phase shift, and prime scan, but high variance is slightly above one at 1.006. These are workload-specific observations, not a claim that either path is always superior.

The late-heavy diagnostic isolates a common interpretation error. The tree records a lower batch p95 than fixed 4 or cost balancing, yet its full wall time is higher than both in Table 2. The path repeatedly waits for complete rounds; batch response behavior alone does not account for idle time, dispatch sequencing, planning, and shutdown. Cost balancing has a higher batch p95 on this pattern but a lower full wall time because its estimated expensive batch starts early. This is a plausible mechanism supported by the implementation and live trace; the saved aggregate data do not isolate one causal component experimentally.

7.2 What this contributes and what it does not

Adaptive self-scheduling, ML-assisted task scheduling, and work-stealing already appear in the cited literature. This prototype's contribution is a reproducible, same-task comparison of a small interpretable online tree, a single-dispatch cost-balancing policy, and a fixed reference while accounting for complete application wall time. The evidence shows the two alternatives have different bottlenecks. A potentially useful combination is a hypothesis for the next stage, not a worldwide-first novelty claim and not a measured Review 2 win.

7.3 Why combine the two paths

The decision tree uses measured pilot behavior rather than relying on task metadata. It beats fixed 4 on prime scan (paired ratio 0.894), but loses on the other five patterns, most on high variance (1.245). Prediction is quick; the current scheduler waits for each round to finish before deciding again. Those barriers may idle workers, though their exact contribution to the loss is unmeasured.

Cost balancing makes one dispatch plan and starts estimated expensive batches early. It wins most on the constructed late-heavy pattern (0.870) and is below 1 on four other patterns. It is slightly slower on high variance (1.006). It cannot correct bad cost estimates from live measurements; wrongly ranked tasks could erase its benefit. That failure mode still needs testing.

The proposed hybrid would use one pilot and one tree prediction to choose a batch-size target, then cost-balance and order only unstarted tasks without repeated round waits. A gate would reject cost estimates that disagree with pilot service times; little remaining work would fall back to fixed dispatch. This aims to retain adaptive sizing and early heavy-task execution while limiting barrier and estimation costs. It may not succeed: no hybrid wall time has been measured.

8 Implementation progress and valid outputs

Six of seven planned modules work: task generation, compiled C execution, tree training, online tree scheduling, cost-balanced scheduling, and paired evaluation. The combined overhead-aware scheduler remains for the next stage. Module count is not a precise measure of effort, but the working prototype exceeds the Review 2 half-complete requirement. No hybrid benchmark is claimed.

Delivered module	Evidence to show	Status
Generator and C kernel	workload.py; native/work_kernel.c; repeatable checksums	Complete
Tree training	training_data.csv; model.json; 73.3 percent held-out accuracy	Complete
Online tree run	live demo trace: chunk 4 to 16; equal checksums	Complete
Cost-balanced run	live demo: late task 187 dispatched first; equal checksums	Complete
Paired evaluator	105 raw rows; 18 summary rows; all groups audited	Complete
Automated checks	12 unit tests passing in Ubuntu	Complete
Combined scheduler	Design and ablation pending	Remaining

8.1 What the present implementation demonstrates

The generator and C kernel execute repeatably; the trainer produces a serialized tree and a separate holdout score; the online scheduler loads that tree and logs chunk changes; the cost planner produces balanced batches and moves the estimated heavy task early; and the evaluator writes raw and summary metrics that can be audited. The terminal evidence shows live tests, a saved-data audit, and a fresh run of both scheduling paths. The screenshots establish that the modules ran in the reported environment, while raw.csv and the source code are needed to inspect the full multi-seed evidence.

The progress count of six delivered modules out of seven is a functional inventory, not a measured percentage of engineering effort. The combined scheduler has no implemented path or measured row in the current paired results. The report therefore separates demonstrations of existing modules from ideas scheduled for later work. A reviewer can trace each table and figure back to the saved output files and compare the claimed method boundary with the source.

9 Challenges and remaining work

Chunk-label accuracy did not predict wall time; the tree's round barrier adds waiting, while cost balancing needs useful task metadata. Synthetic tasks and one two-worker WSL2 host limit generalization.

Next, test an overhead-gated combined scheduler: use the tree after a pilot, balance only unstarted tasks when metadata are informative, and remove unnecessary round barriers. Compare fixed 4, tree, cost balance and the combined path on matched seeds. Then perturb cost estimates, test more workers and longer tasks, report uncertainty intervals, and repeat on another Linux machine or real trace. Keep the combined path only if full wall time improves with equal outputs.

9.1 A testable next-stage design

The next implementation would run a short pilot, compute the existing five tree features, and make one tree prediction for the remaining work. It would translate the predicted chunk into a target batch cost, group unstarted tasks by estimated cost, and submit the heaviest batches first without another full-round wait. Before using that plan, it would compare pilot task-cost estimates with measured service times and check whether enough work remains to repay planning overhead. If either check fails, it would dispatch the remaining tasks with fixed chunk 4. The agreement and overhead thresholds must be set on development runs, not adjusted after seeing the final test results.

A valid test would run the proposed combined path beside fixed 4, tree-only, and balance-only under the same generated task lists and shuffled method order. It should report checksum equality, full wall time and paired ratios first, then use startup, queue wait, service, and batch-tail measures to explain the result. Separate ablations should remove the gate or the balancing step to show whether either part contributes. More workers, larger task counts, perturbed cost estimates, a second Linux host, and a non-synthetic workload would test how far the conclusion travels. None of those future measurements is inserted into the present result tables.

9.2 Practical limitations

The six generated patterns cannot represent arbitrary OS workloads, and the late-heavy pattern was chosen precisely because a metadata-based ordering may help it. Only two worker processes and one host were used; scaling and NUMA effects are unmeasured. The prime kernel is a useful distinct computation but still not a trace from an external application. The cost estimator can be wrong, and the tree's profiling labels are noisy and only 73.3 percent exact on holdout. These limitations make the most defensible conclusion narrow: the current implementation works and exposes a meaningful tradeoff, but general speedup and the value of a hybrid remain unproven.

Terminal results

The WSL project has a logical directory alias at the prompt shown below. The screenshots are cropped for print readability; the white-on-black text is the original terminal display.

```
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$ python3 -m unittest discover -s tests -q
----------------------------------------------------------------------
Ran 12 tests in 0.365s

OK
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$
```

Figure A1 Fresh Linux unit-test run: 12 tests, OK.

```
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$ python3 -m os_sched.verify_paired
PASS  12-column raw data and 18 summary rows audited
PASS  35 paired input groups / 105 method runs
PASS  all three checksums equal within every group
PASS  summary wall times and paired ratios recomputed

Median paired wall-time ratio to fixed chunk 4 (lower is better)
workload        n     tree  balanced
uniform         5    1.047     0.982
moderate        5    1.025     0.956
high            5    1.245     1.006
phase_shift     5    1.097     0.949
prime_scan      5    0.894     0.917
late_heavy     10    1.046     0.870
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$
```

Figure A2 Saved benchmark audit: 35 paired groups, 105 runs, equal checksums and recalculated ratios.

```
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$ bash show_both_review2.sh
Tree scheduler (first decisions and final check):
LIVE prime_scan tasks=192 workers=2 model=results_final_linux/model.json
round=01 chunk=4->16 completed=16 decision_us=22.2
round=01 chunk=4->4 completed=16 decision_us=17.7
END fixed4=0.182s tree=0.238s guarded=0.184s checksums_equal=True output=/tmp/review2_tree.json
Cost-balanced scheduler (late-heavy run):
Tail spike at task 187; balanced scheduler moves its batch to dispatch position 0
fixed4=0.479s pool_map16=0.517s balanced32=0.427s checksums_equal=True
sash@Minato:/mnt/c/Users/SASHWATH/Documentswork/review2-project$
```

Figure A3 Fresh live runs of both paths: the tree changes chunk 4 to 16; cost balancing dispatches the late task first. Both pass checksum checks. The tree trace is filtered to its first decisions and final result.

Evidence files and reproduction notes

The screenshots above serve different purposes. Figure A1 records a fresh twelve-test Linux run; it checks functions and invariants, not scheduler speed. Figure A2 runs the saved-data audit, which counts 35 matched input groups and 105 timed method rows, verifies checksum equality, and recalculates the reported paired ratios. Figure A3 shows two live demonstrations, so its elapsed seconds are not substituted for the multi-seed medians in Tables 1 and 2. The figures make the outputs easy to show during review, while the CSV and JSON files provide the inspectable records behind the result claims.

For the tree trace, the first decision begins at chunk 4 and selects chunk 16 after the 16-task pilot. The screenshot includes both the tree and guarded-tree first decisions, plus an END line that confirms equal checksums against fixed 4. The balanced trace reports that late task 187 was moved to dispatch position 0 and again confirms equal checksums. These are observations from the particular live seeds shown, not an effect size. The paired study instead evaluates the declared seed ranges and reports medians over five or ten matched trials.

The repository keeps the executable source in native/ and os_sched/, automated checks in tests/, the trained model in results_final_linux/model.json, and the Review 2 records in results_paired_both_review/. In that last directory raw.csv contains workload, seed, method order, elapsed time, checksum, batch p95, startup, and other timing fields for each run. summary.csv reduces those rows to workload-by-method medians; manifest.json records the scope and timing boundary. The PDF and editable Word report are in output/. The repository link below leads to those artifacts and their source.

A rebuild compiles the C library before Python tests or experiments. A fresh run of the paired experiment may produce different seconds because WSL2 and the host schedule processes under changing load; it should nevertheless preserve workload definitions, method set, checksum equality, row counts, and the audit procedure. Claims in this report refer to the saved verified run, not to an assurance that every rerun will keep the same ranking. This is why the exact task seeds, method ordering rule, raw measurements, and verifier are published together.

References

Booth, J. D., & Lane, P. A. (2022). An adaptive self-scheduling loop scheduler. Concurrency and Computation: Practice and Experience, 34(6), e6750. https://doi.org/10.1002/cpe.6750

Nookala, P., Chard, K., & Raicu, I. (2024). X-OpenMP: eXtreme fine-grained tasking using lock-less work stealing. Future Generation Computer Systems, 159, 444-458. https://doi.org/10.1016/j.future.2024.05.019

Qawasmeh, A., Malik, A. M., & Chapman, B. M. (2016). Adaptive OpenMP task scheduling using runtime APIs and machine learning. In Proceedings of the 2015 IEEE 14th International Conference on Machine Learning and Applications (pp. 889-895). https://doi.org/10.1109/ICMLA.2015.111

Project repository: https://github.com/Sashwath-sash/ml-adaptive-chunk-scheduling